Redes Neuronales

Antonio Falcó y Juan Pardo

Introducción a la Inteligencia Artificial

Aprendizaje

Una breve introducción al concepto de aprendizaje y a la inteligencia artificial en el campo del diseño de producto y de los procesos de fabricación.

El proceso de aprendizaje

El aprendizaje es el proceso de adquisición de conocimientos, habilidades y actitudes a través de la experiencia, la instrucción y el estudio.

El aprendizaje en la inteligencia artificial

En el campo de la inteligencia artificial, el aprendizaje es el proceso de entrenar con un ordenador un modelo matemático alimentado con datos para realizar una tarea específica, como reconocer patrones en datos, tomar decisiones o predecir resultados.

El proceso de aprendizaje con imágenes digitales

En el campo del procesamiento de imágenes digitales, el aprendizaje es el proceso de entrenar con un ordenador un modelo matemático alimentado con datos para analizar y procesar imágenes digitales, como reconocer objetos, segmentar regiones de interés o mejorar la calidad de las imágenes.

Aprendizaje

El esquema del proceso de aprendizaje máquina.

$$\boxed{\text{Entrenamiento} = \text{Aprendizaje}}$$

$$\text{Datos} = (\text{Imágenes}, \text{Etiquetas})$$

$$\boxed{\text{Modelo}} \times \text{Imágenes} \longrightarrow \text{Etiquetas}$$

$$\boxed{\text{Procedimiento} = \text{Algoritmo}}$$

$$\boxed{\text{Modelo Entrenado}}$$

$$\text{Imágenes} \longrightarrow \text{Etiquetas}$$

- El proceso de aprendizaje = entrenamiento lleva consigo la construcción de un modelo matemático que es alimentado con datos (imágenes y etiquetas) para realizar una tarea específica, como reconocer objetos en imágenes, segmentar regiones de interés o mejorar la calidad de las imágenes.
- La construcción del modelo se realiza empleando un procedimiento = algoritmo.

Problemas clave

1. **Representación de datos** : La representación de datos es un desafío importante, ya que los datos pueden ser complejos y difíciles de analizar.
2. **Optimización** : La optimización es un problema clave en la inteligencia artificial, ya que muchos algoritmos buscan encontrar las mejores soluciones entre múltiples opciones.
3. **Escalabilidad** : La escalabilidad se refiere a la capacidad de los algoritmos para manejar grandes cantidades de datos y hacer predicciones precisas en tiempo real.
4. **Interpretabilidad** : La interpretabilidad es el problema de entender cómo los algoritmos llegan a sus conclusiones y por qué toman ciertas decisiones.
5. **Robustez** : La robustez se refiere a la capacidad de los algoritmos para funcionar correctamente en diferentes condiciones y ambientes.
6. **Explicabilidad** : La explicabilidad es el problema de justificar las decisiones tomadas por los algoritmos y cómo se relacionan con los datos de entrada.
7. **Equidad** : La equidad se refiere a la capacidad de los algoritmos para tomar decisiones no discriminatorias y respetuosas con los derechos humanos.

Ejemplo de una red neuronal simple

Objetivo : Diseñar una red neuronal simple que pueda predecir si tipo de flor es *Iris setosa* a partir de la anchura y la longitud (cm) del pétalo de una flor *Iris*.

$$\boxed{\text{Entrenamiento} = \text{Aprendizaje}}$$

$$\text{Datos} = ((\text{Anchura},\text{Longitud}),\text{Tipo})$$

$$\boxed{\text{Modelo}} \times (\text{Anchura},\text{Longitud}) \longrightarrow \text{Tipo}$$

$$\boxed{\text{Procedimiento} = \text{Algoritmo}}$$

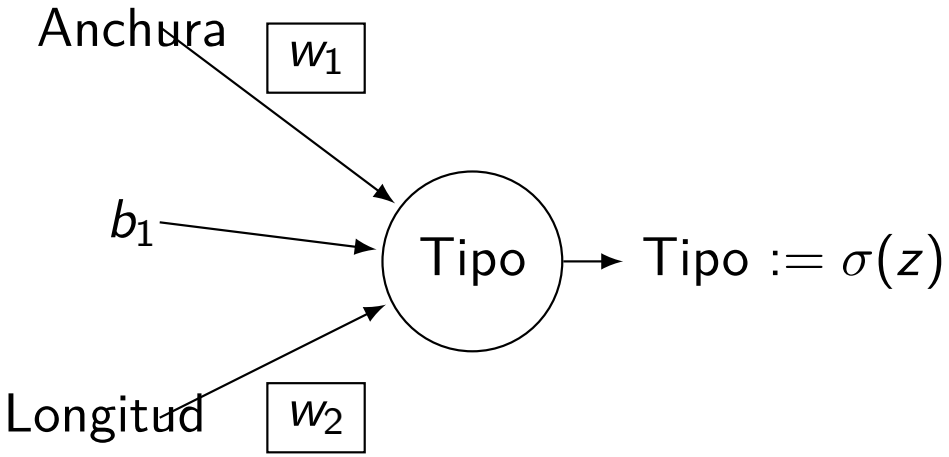
$$\boxed{\text{Modelo Entrenado}}$$

$$(\text{Anchura},\text{Longitud}) \longrightarrow \text{Tipo}$$

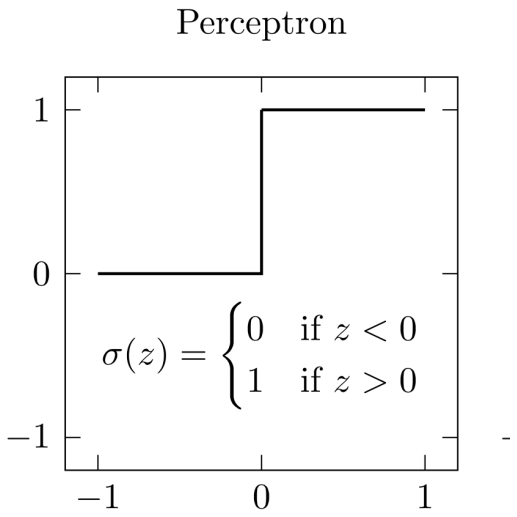
Modelo : $\text{Tipo} = \sigma\left(w_1 \times \text{Anchura} + w_2 \times \text{Longitud} + b_1\right) =$

$$\begin{cases} 0 & \text{si Tipo} = \text{Iris setosa} \\ 1 & \text{si Tipo} \neq \text{Iris setosa} \end{cases}$$

Modelo del perceptrón



Modelo del perceptrón



Consideramos la función de activación que caracteriza el tipo de flor *Iris setosa* como

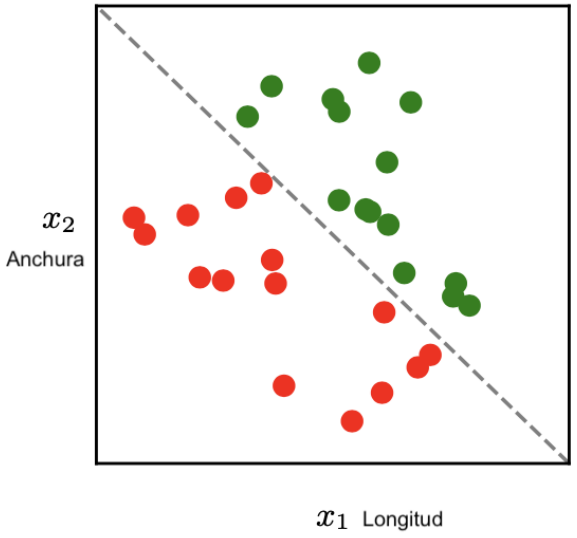
$$\sigma(z) = \begin{cases} 0\,(\textit{Iris setosa}) & \text{si } z < 0 \\ 1\,(\textit{No Iris setosa}) & \text{si } z > 0 \end{cases}$$

y tomamos

$$z = w_1 \times \text{Anchura} + w_2 \times \text{Longitud} + b_1$$

La caracterización del modelo de perceptrón se reduce al conocimiento de los parámetros w_1, w_2 (pesos) y b_1 (sesgo).

Modelo del perceptrón



La separación de las flores *Iris setosa* y *No Iris setosa* se caracteriza empleando la ecuación

$$z = w_1 \times \text{Anchura (cm)} + w_2 \times \text{Longitud (cm)} + b_1 = 0$$

en la imagen se representa con la línea discontinua.

Modelo del perceptrón : Aprendizaje computacional

Objetivo : Tenemos que calcular los parámetros w_1, w_2 y b_1 que caracterizan el modelo de perceptrón. Para ello, empleamos el algoritmo de aprendizaje computacional que se basa en la minimización de la función de coste siguiente :

- Tomamos un conjunto de datos de entrenamiento

$$\{(x^{(1)}, y^{(1)}), \ldots, (x^{(m)}, y^{(m)})\},$$

  donde $x^{(i)} = (x_1^{(i)}, x_2^{(i)})$, siendo $x_1^{(i)} =$ Longitud y $x_2^{(i)} =$ Anchura , es el vector de características de la i-ésima flor de la base de datos y $y^{(i)} \in \{0, 1\}$ es la etiqueta correspondiente.

- La función de coste se puede definir como :

$$\frac{1}{m}\sum_{i=1}^{m}(y^{(i)} - \sigma(z^{(i)}))^2 = \frac{1}{m}\sum_{i=1}^{m}(y^{(i)} - \sigma(w_1 \times x_1^{(i)} + w_2 \times x_2^{(i)} + b_1))^2,$$

Modelo del perceptrón : Aprendizaje computacional

- Analicemos La función de coste :

$$0 \leq \frac{1}{m}\sum_{i=1}^{m}(y^{(i)} - \sigma(w_1 \times x_1^{(i)} + w_2 \times x_2^{(i)} + b_1))^2 \leq 1,$$

  En el caso de que $y^{(i)} \neq \sigma(z^{(i)})$, para todos los datos de entrenamiento la función de coste tomaría el valor de 1 y 0 cuando $y^{(i)} = \sigma(z^{(i)})$.

- Supongamos fijados los valores de w_1, w_2 y b_1, y observemos que cada vez que nos equivocamos en la predicción, la función de coste aumenta una cantidad $1/m$, luego el valor de la función de coste nos indica el porcentaje de errores en la predicción :

$$\frac{\text{Cantidad de errores en la predicción}}{\text{Número total de datos de entrenamiento}}(w_1, w_2, b_1).$$

Modelo del perceptrón : Aprendizaje computacional

- Nuestro objetivo es minimizar la función de coste :

$$\min_{w_1, w_2, b_1} \frac{\text{Cantidad de errores en la predicción}}{\text{Número total de datos de entrenamiento}}(w_1, w_2, b_1),$$

esto es, minimizar el porcentaje de errores en la predicción para los datos de entrenamiento :

$$\min_{w_1, w_2, b_1} \frac{1}{m} \sum_{i=1}^{m} (y^{(i)} - \sigma(w_1 \times x_1^{(i)} + w_2 \times x_2^{(i)} + b_1))^2.$$

- El resultado de este proceso de minimización nos proporcionará los valores de los parámetros $\widehat{w}_1, \widehat{w}_2, \widehat{b}_1$ que caracterizan el modelo de perceptrón.

Modelo entrenado del perceptrón

- El modelo entrenado de perceptrón viene dado por la función : :

  $$\text{Mod. Entr.}(\text{Longitud}, \text{Anchura}) = \sigma(\widehat{w}_1 \times \text{Longitud} + \widehat{w}_2 \times \text{Anchura} + \widehat{b}_1).$$

- Podemos emplear un segundo conjunto de datos de prueba para evaluar la precisión del modelo entrenado.

Código Python

Este proyecto requiere Python 3.7 o superior, Scikit-Learn 1.0.1 o superior y TensorFlow 2.8.0 o superior. :

```
import sys
assert sys.version_info >= (3, 7)
from packaging import version
import sklearn
assert version.parse(sklearn.__version__) >= version.parse
  ("1.0.1")
import tensorflow as tf
assert version.parse(tf.__version__) >= version.parse("
  2.8.0")
```

Código Python

Definimos las fuentes para tener una mejor visualización de los gráficos :

```
import matplotlib.pyplot as plt
plt.rc('font', size=14)
plt.rc('axes', labelsize=14, titlesize=14)
plt.rc('legend', fontsize=14)
plt.rc('xtick', labelsize=10)
plt.rc('ytick', labelsize=10)
```

Código Python

Creamos un directorio para guardar las imágenes generadas :

```
from pathlib import Path
IMAGES_PATH = Path()/"images"/"ann"
IMAGES_PATH.mkdir(parents=True, exist_ok=True)

def save_fig(fig_id, tight_layout=True,fig_extension="png",
    resolution=300):
  path = IMAGES_PATH / f"{fig_id}.{fig_extension}"
  if tight_layout:
      plt.tight_layout()
  plt.savefig(path, format=fig_extension, dpi=resolution)

```

Código Python

Definimos el perceptron :

```
import numpy as np
from sklearn.datasets import load_iris
from sklearn.linear_model import Perceptron

iris = load_iris(as_frame=True)
X = iris.data[["petal length (cm)", "petal width (cm)"]].
    values
y = (iris.target == 0)  # Iris setosa

per_clf = Perceptron(random_state=42)
per_clf.fit(X, y)

```

Código Python

Empleamos el modelo para determinar si las flores son de tipo *Iris setosa* o no :

```
X_new = [[2, 0.5], [3, 1]]
y_pred = per_clf.predict(X_new)  # predice True y False para
      estas 2 flores

print(y_pred)

```

[True False]

Código Python

El ‘Perceptrón‘ es equivalente a un ‘SGDClassifier‘ con ‘loss="perceptron"‘, sin regularización y una tasa de aprendizaje constante igual a 1 :

```
# codigo extra - muestra como construir y entrenar un Perceptron
from sklearn.linear_model import SGDClassifier

sgd_clf = SGDClassifier(loss="perceptron", penalty=None, learning_rate="constant", eta0=1, random_state=42)
sgd_clf.fit(X, y)
assert (sgd_clf.coef_ == per_clf.coef_).all()
assert (sgd_clf.intercept_ == per_clf.intercept_).all()
print(sgd_clf.predict([[2, 0.5], [3, 1]]))

```

[True False]

Código Python

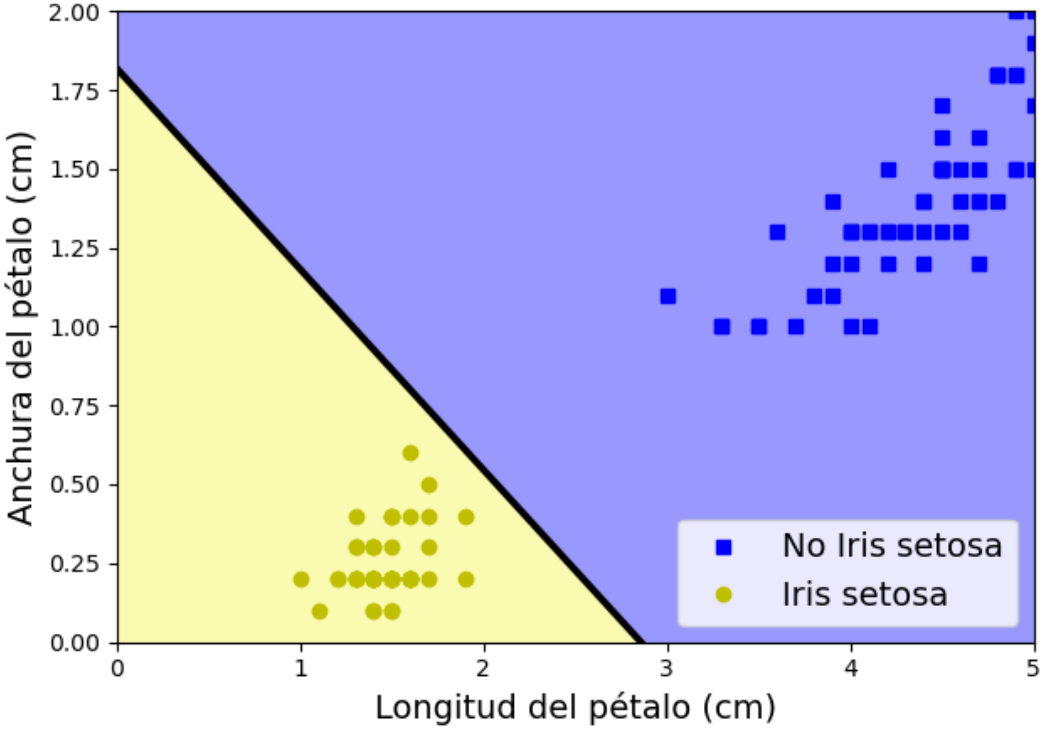
Cuando el perceptrón encuentra un límite de decisión que separa adecuadamente las clases, deja de aprender. Esto significa que el límite de decisión suele estar bastante cerca de una clase :

```
# extra code - plots the decision boundary of a Perceptron
    on the iris dataset

import matplotlib.pyplot as plt
from matplotlib.colors import ListedColormap

a = -per_clf.coef_[0, 0] / per_clf.coef_[0, 1]
b = -per_clf.intercept_ / per_clf.coef_[0, 1]
axes = [0, 5, 0, 2]
x0, x1 = np.meshgrid(
    np.linspace(axes[0], axes[1], 500).reshape(-1, 1),
    np.linspace(axes[2], axes[3], 200).reshape(-1, 1),
)
X_new = np.c_[x0.ravel(), x1.ravel()]
y_predict = per_clf.predict(X_new)
zz = y_predict.reshape(x0.shape)
custom_cmap = ListedColormap(['#9898ff', '#fafab0'])
```

Código Python

```
plt.figure(figsize=(7, 3))
plt.plot(X[y == 0, 0], X[y == 0, 1], "bs", label="No Iris setosa")
plt.plot(X[y == 1, 0], X[y == 1, 1], "yo", label="Iris setosa")
plt.plot([axes[0], axes[1]], [a * axes[0] + b, a * axes[1] + b], "k-",
         linewidth=3)
plt.contourf(x0, x1, zz, cmap=custom_cmap)
plt.xlabel("Longitud del petalo (cm)")
plt.ylabel("Anchura del petalo (cm)")
plt.legend(loc="lower right")
plt.axis(axes)
plt.show()
```

Figura


2.00
1.75
1.50
1.25
1.00
0.75
0.50
0.25
0.00
Anchura del pétalo (cm)
0
1
2
3
4
5
Longitud del pétalo (cm)
No Iris setosa
Iris setosa

Funciones de activación

- Un parámetro más de este tipo de modelos es la elección de la función de activación. La función de activación es una función matemática que se aplica a la salida de cada neurona en la red neuronal.
- En el caso particular del perceptrón, la función de activación es la función de Heaviside :

$$\sigma(z) = H(z) = \begin{cases} 0 & \text{si } z < 0, \\ 1 & \text{si } z > 0. \end{cases}$$

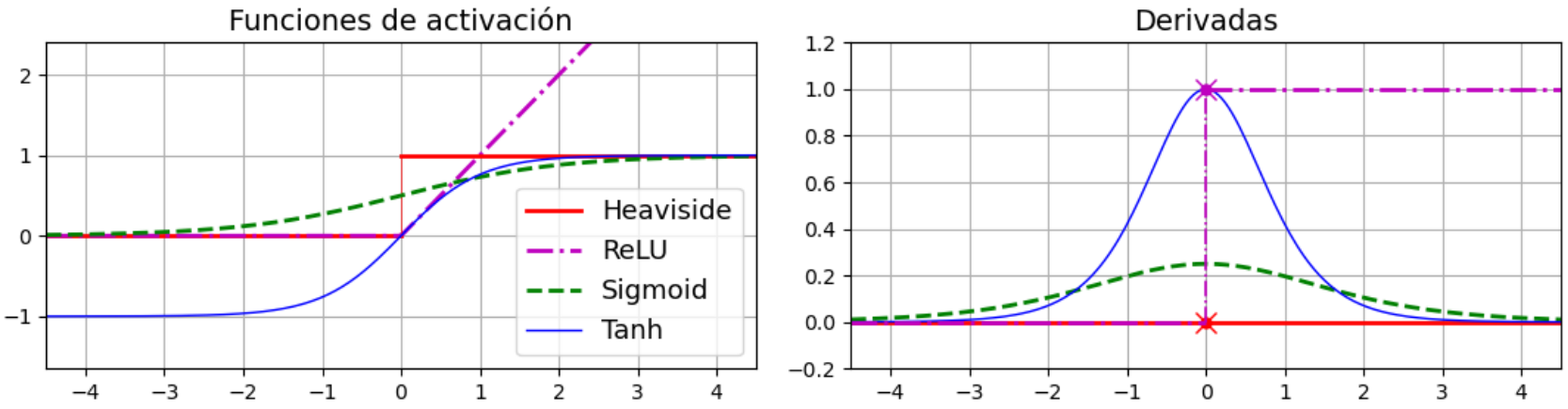
- Otras funciones comunmente empleadas son la función sigmoide, la función tangente hiperbólica y la función ReLU :

$$\sigma(z) = \frac{1}{1 + e^{-z}}, \quad \sigma(z) = \tanh(z), \quad \sigma(z) = \max(0, z).$$

Recordar que

$$\tanh(z) = \frac{e^z - e^{-z}}{e^z + e^{-z}}$$

Figura


Funciones de activación
Heaviside
ReLU
Sigmoid
Tanh
Derivadas

Funciones de coste

- Otro parámetro más de este tipo de modelos es la elección de la función de coste. La función de coste es una función matemática que se emplea para medir la diferencia entre la salida predicha por el modelo y la salida real de los datos de entrenamiento.
- En nuestro caso, la función de coste es la función de coste cuadrático medio :

$$\frac{1}{m}\sum_{i=1}^{m}(y^{(i)} - \sigma(w_1 \times x_1^{(i)} + w_2 \times x_2^{(i)} + b_1))^2$$

- La función que relaciones los datos de entrada con la salida que deseamos predecir se denomina función de hipótesis y la denotamos por $z = \phi_{\text{modelo}}(x_1, x_2)$:
- La red neuronal se puede ver como una función $z = \phi^{\sigma}_{(w_1,w_2,b_1)}(x_1, x_2) = \sigma(w_1 \times x_1 + w_2 \times x_2 + b_1)$.

Funciones de coste

- Podemos entonces escribir la función de pérdida como :

$$\frac{1}{m}\sum_{i=1}^{m}(\phi_{\text{modelo}}(x_1^{(i)}, x_2^{(i)}) - \phi^{\sigma}_{(w_1,w_2,b_1)}(x_1^{(i)}, x_2^{(i)}))^2$$

- Aunque se pueden en general definir funciones de coste más complejas, la función de coste cuadrático medio es una de las más comunes en la práctica.
- La elección de la función de coste es un aspecto importante en el diseño de un modelo de red neuronal, ya que afecta a la forma en que el modelo aprende de los datos de entrenamiento.
- En general una función de pérdida se puede escribir como :

$$\mathcal{L}(\phi_{\text{modelo}}(x_1, x_2), \phi^{\sigma}_{(w_1,w_2,b_1)}(x_1, x_2), \text{Datos Entrenamiento}).$$

- Con una notación más compacta, la función de pérdida se puede escribir como :

$$\mathcal{L}(\mathbf{y}, \phi^{\sigma}_{\boldsymbol{\theta}}(\mathbf{x})),$$

donde $(\mathbf{x}, \mathbf{y})$ son datos del conjunto de entrenamiento y $\boldsymbol{\theta}$ son los parámetros del modelo.

Funciones de coste

- Con una notación más compacta, la función de pérdida se puede escribir como :

$$\mathcal{L}(\mathbf{y}, \phi^{\sigma}_{\boldsymbol{\theta}}(\mathbf{x})),$$

  donde $(\mathbf{x}, \mathbf{y})$ son datos del conjunto de entrenamiento y $\boldsymbol{\theta}$ son los parámetros del modelo.
- El modelo que obtenemos al minimizar la función de pérdida se denomina modelo entrenado y formalmente se representa por

$$\mathbf{y} = \phi^{\sigma}_{\widehat{\boldsymbol{\theta}}}(\mathbf{x})$$

  donde $\widehat{\boldsymbol{\theta}}$ son los parámetros del modelo entrenado que formalmente se obtienen al resolver el problema de optimización :

$$\widehat{\boldsymbol{\theta}} = \arg\min_{\boldsymbol{\theta}} \mathcal{L}(\mathbf{y}, \phi^{\sigma}_{\boldsymbol{\theta}}(\mathbf{x})).$$

Modelos de Redes Neuronales

- Los modelos de redes neuronales son modelos matemáticos que se inspiran en la forma en que funcionan las neuronas en el cerebro humano.
- La idea es la siguiente. Consideremos ahora un conjunto de datos de entrada $\mathbf{x} = (x_1, x_2, \ldots, x_n)$ y un conjunto de datos de salida $\mathbf{y} = (y_1, y_2, \ldots, y_m)$ **que representan objetos que podemos codificar empleando vectores numéricos**.

Modelos de Redes Neuronales : Una capa

Se elige una función de activación σ y se define la función de hipótesis $\phi^{\sigma}_{\boldsymbol{\theta}}(\mathbf{x})$ como :

1. Para cada par de datos $(\mathbf{x}, \mathbf{y})$ del conjunto de entrenamiento, procedemos del modo siguiente :
2. Escribimos $\mathbf{z}_0 := \mathbf{x}$, y consideramos como vector de parámetros $\boldsymbol{\theta} = (W_1, \mathbf{b}_1)$ del modelo una matriz de pesos W_1 y un vector de sesgos $\mathbf{b}_1$ siguiente :

$$W_1 = \begin{bmatrix} W_{11}^{(1)} & W_{12}^{(1)} & \cdots & W_{1n}^{(1)} \\ W_{21}^{(1)} & W_{22}^{(1)} & \cdots & W_{2n}^{(1)} \\ \vdots & \vdots & \ddots & \vdots \\ W_{m1}^{(1)} & W_{N_1 2}^{(1)} & \cdots & W_{mn}^{(1)} \end{bmatrix}, \quad \mathbf{b}_1 = \begin{bmatrix} b_1^{(1)} \\ b_2^{(1)} \\ \vdots \\ b_m^{(1)} \end{bmatrix},$$

Modelos de Redes Neuronales : Una capa

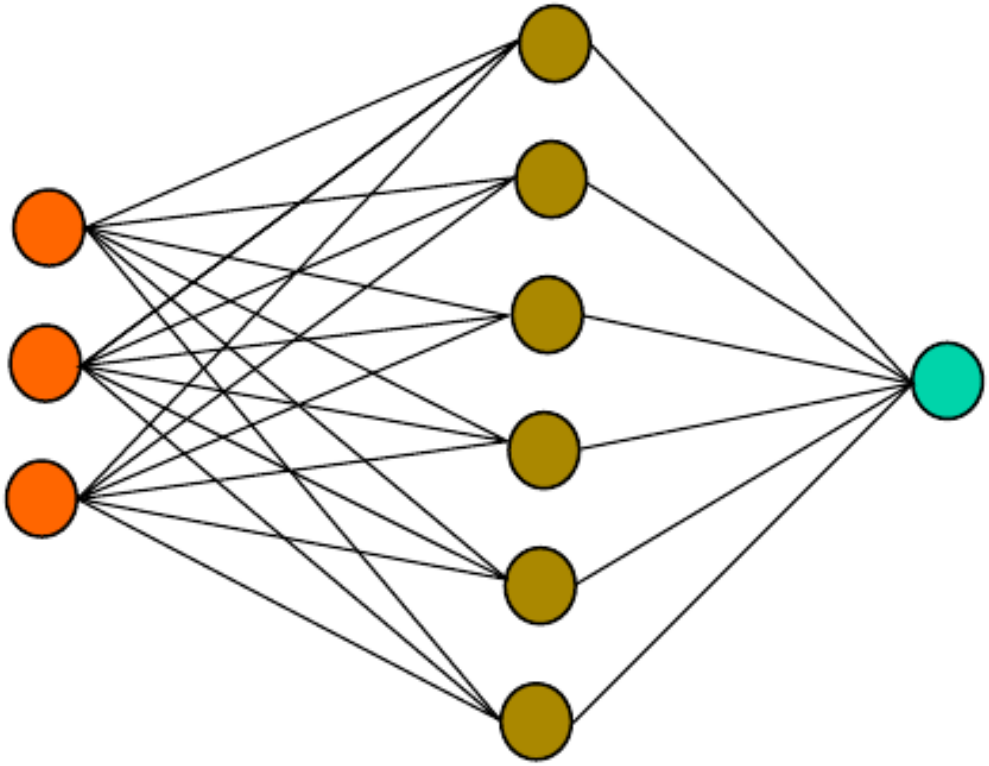
Calculamos un vector $\mathbf{z}_1$ del modo siguiente :

$$\mathbf{z}_1 = \sigma(W_1\mathbf{z}_0 + \mathbf{b}_1) := \begin{bmatrix} \sigma(\sum_{j=1}^{n} W_{1j}^{(1)} x_j + b_1^{(1)}) \\ \sigma(\sum_{j=1}^{n} W_{2j}^{(1)} x_j + b_2^{(1)}) \\ \vdots \\ \sigma(\sum_{j=1}^{n} W_{mj}^{(1)} x_j + b_m^{(1)}) \end{bmatrix}$$

El vector de salida $\mathbf{z}_1$ tiene el mismo número de componentes que el vector de etiquetas $\mathbf{y}$. Podemos considerar $\mathbf{z}_1$ como un nuevo vector de características que se emplea para calcular la salida del modelo. El error cometido en la predicción se puede medir empleando :

$$\|\mathbf{z}_1 - \mathbf{y}\|^2 := \sum_{i=1}^{m} \left(\sigma \left(\sum_{j=1}^{n} W_{ij}^{(1)} x_j + b_j^{(1)} \right) - y_i \right)^2 .$$

Modelos de Redes Neuronales : Una capa



En esta imagen representa una red neuronal de una capa con $n = 3$, es decir $\mathbf{x} = (x_1, x_2, x_3)$, y $m = 6$, es decir $\mathbf{y} = (y_1, y_2, y_3, y_4, y_5, y_6)$. En consecuencia la matriz W_1 tiene dimensión $6 \times 3 = 18$ y el vector $\mathbf{b}_1$ tiene dimensión 6. Esto hace un total de 24 parámetros que caracterizan el modelo.

Modelos de Redes Neuronales : Una capa

La función de coste para un conjunto de entrenamiento

$$(\mathbf{x}_1, \mathbf{y}_1), \ldots, (\mathbf{x}_\ell, \mathbf{y}_\ell)$$

donde para cada dato de entrada $\mathbf{x}_1, \mathbf{x}_2, \ldots, \mathbf{x}_\ell$ puedo calcular un vector de salida $\mathbf{z}_1^{(1)}, \mathbf{z}_1^{(2)}, \ldots \mathbf{z}_1^{(\ell)}$ del modo siguiente :

$$\begin{aligned}
\mathbf{z}_1^{(1)} &= \sigma(W_1\mathbf{x}_1 + \mathbf{b}_1), \\
\mathbf{z}_1^{(2)} &= \sigma(W_1\mathbf{x}_2 + \mathbf{b}_1), \\
\vdots \quad & \qquad \vdots \\
\mathbf{z}_1^{(\ell)} &= \sigma(W_1\mathbf{x}_\ell + \mathbf{b}_1).
\end{aligned}$$

se puede escribir para cada dato $\mathbf{x}_i = (x_1^{(i)}, \ldots, x_n^{(i)})$ e $\mathbf{y}_i = (y_1^{(i)}, \ldots, y_m^{(i)})$ con $1 \leq i \leq \ell$, como

$$\frac{1}{\ell}\sum_{i=1}^{\ell} \|\mathbf{z}_1^{(i)} - \mathbf{y}_i\|^2 = \frac{1}{\ell}\sum_{i=1}^{\ell}\sum_{j=1}^{m}\left(\sigma\left(\sum_{k=1}^{n} W_{jk}^{(1)} x_k^{(i)} + b_j^{(1)}\right) - y_j^{(i)}\right)^2.$$

Modelos de Redes Neuronales : Una capa

La función de coste para el conjunto de entrenamiento

$$\mathcal{L}(W_1, \mathbf{b}_1) = \frac{1}{\ell} \sum_{i=1}^{\ell} \sum_{j=1}^{m} \left(\sigma \left(\sum_{k=1}^{n} W_{jk}^{(1)} x_k^{(i)} + b_j^{(1)} \right) - y_j^{(i)} \right)^2$$

depende de los parámetros W_1 y $\mathbf{b}_1$. El objetivo es minimizar la función de coste $\mathcal{L}(W_1, \mathbf{b}_1)$ respecto a los parámetros W_1 y $\mathbf{b}_1$. Una vez calculados los valores óptimos $(\widehat{W}_1, \widehat{b}_1)$ la función predicción se puede escribir como :

$$\mathbf{y} = \phi^{\sigma}_{\widehat{W}_1, \widehat{\mathbf{b}}_1}(\mathbf{x}) = \sigma(\widehat{W}_1 \mathbf{x} + \widehat{\mathbf{b}}_1).$$